



Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

Lab 3: Implementation of Activation Functions

Objective:

- To implement and compare Sigmoid, Tanh, and ReLU to understand their impact on model convergence and the importance of non-linearity.
- To build a simple feedforward neural network and see how the choice of activation function in the hidden layers changes the network's raw output.

Software and Tools:

1.6 GHz or faster processor
4 GB of RAM
20 GB of available hard disk space
Windows 7 or later
Google Colab

Colab Notebook for Lab 1:

<https://colab.research.google.com/drive/1Rna7swh8K-thtYh-Og0Hj36aI3eapuDE?usp=sharing>

Students should open the notebook, follow instructions, and perform the tasks to complete the lab exercises.

Background and Procedure:

A Feedforward Neural Network (FNN) is the simplest type of Artificial Neural Network (ANN). It is called “feedforward” because data flows in one direction, from the input layer to the output layer, without looping back.

FNNs are the foundation of deep learning models, and they are widely used for tasks like classification, regression, and pattern recognition.

Structure of FNN:

(a) Input Layer

- This is the first layer of the network.
- It receives the features (independent variables) of the dataset.
- Example (Iris dataset):
 - Input features = sepal length, sepal width, petal length, petal width
 - Total 4 neurons in input layer.



Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

(b) Hidden Layers

- Located between the input and output layers.
- Each neuron in the hidden layer applies two steps:

1. Weighted sum of inputs + bias:

$$z = W \cdot x + b \text{ where:}$$

W = weight matrix (strength of connections between neurons)

x = input vector

b = bias (shifts the activation function)

2. Activation function:

Non-linear transformation applied to z.

Common functions:

Sigmoid: $f(z) = \frac{1}{1+e^{-z}}$

Tanh: $f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

ReLU (Rectified Linear Unit): $f(z) = \max(0, z)$

Without activation functions, the model would only learn linear relationships. Non-linearity allows the network to learn complex patterns.

(c) Output Layer

- Produces the final prediction.
- The number of output neurons depends on the task:
 - Regression: 1 neuron (e.g., predicting house price).
 - Binary classification: 1 neuron with sigmoid activation.
 - Multi-class classification: k neurons (where k = number of classes).

Example (Iris dataset): 3 neurons (Setosa, Versicolor, Virginica).

Forward Pass: The data flows forward through the layers.

Mathematically:

$$x \rightarrow (W_1x + b_1) \rightarrow f(\cdot) \rightarrow (W_2h + b_2) \rightarrow \text{Output}$$



Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

Where:

- x = input vector (features)
- W, b = weights and biases
- $f(\cdot)$ = activation function
- h = hidden layer representation

This process generates predicted output ($y^{\wedge}$).

Import libraries

Torch: PyTorch main library.

torch.nn: helps us build neural networks (layers, modules).

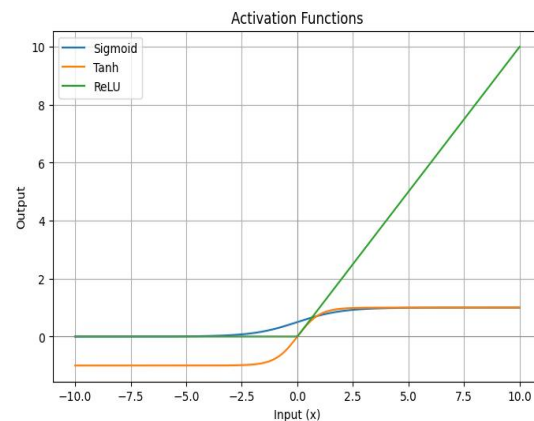
torch.nn.functional as F: provides functions like activation functions (relu, softmax).

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
%matplotlib inline

# Sample input values ranging from -10 to 10
x = torch.linspace(-10, 10, 200)

sigmoid_y = torch.sigmoid(x)
tanh_y = torch.tanh(x)
relu_y = F.relu(x)

plt.figure(figsize=(8, 5))
plt.plot(x, sigmoid_y, label='Sigmoid')
plt.plot(x, tanh_y, label='Tanh')
plt.plot(x, relu_y, label='ReLU')
plt.axhline(0, color='gray', linewidth=0.5)
plt.axvline(0, color='gray', linewidth=0.5)
plt.title('Activation Functions')
plt.xlabel('Input (x)')
plt.ylabel('Output')
plt.legend()
plt.grid(True)
plt.show()
```



Create a Custom Neural Network Class

Create a custom model class that is inherited from `nn.Module`. This model will have two hidden layers and an output layer.



Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

Create an instance of a model

Instantiate the model and print it to see its structure.

Sigmoid Activation

```
class ModelSigmoid(nn.Module):
    #Input layer (4 features of the flower)
    #Hidden Layer 1 (number of neurons)
    #H2 (n)
    #Output (3 Classes of IRIS)
    def __init__(self, in_features=4, h1=8, h2=9, out_features=3):
        super().__init__() #initiate our nn model
        self.fc1 = nn.Linear(in_features, h1)
        self.fc2 = nn.Linear(h1, h2)
        self.out = nn.Linear(h2, out_features)

    def forward(self, x):
        x = torch.sigmoid(self.fc1(x))    # Hidden Layer 1
        x = torch.sigmoid(self.fc2(x))    # Hidden Layer 2
        x = self.out(x)                   # Output logits
        return x
```

```
torch.manual_seed(32)
model_sigmoid = ModelSigmoid()
print(model_sigmoid)
```

```
ModelSigmoid(
  (fc1): Linear(in_features=4, out_features=8, bias=True)
  (fc2): Linear(in_features=8, out_features=9, bias=True)
  (out): Linear(in_features=9, out_features=3, bias=True)
)
```

Tanh Activation

```
class ModelTanh(nn.Module):
    #Input layer (4 features of the flower)
    #Hidden Layer 1 (number of neurons)
    #H2 (n)
    #Output (3 Classes of IRIS)
    def __init__(self, in_features=4, h1=8, h2=9, out_features=3):
        super().__init__() #initiate our nn model
        self.fc1 = nn.Linear(in_features, h1)
        self.fc2 = nn.Linear(h1, h2)
        self.out = nn.Linear(h2, out_features)

    def forward(self, x):
        x = torch.tanh(self.fc1(x))    # Hidden Layer 1
        x = torch.tanh(self.fc2(x))    # Hidden Layer 2
        x = self.out(x)
        return x
```

```
torch.manual_seed(32)
model_tanh = ModelTanh()
print(model_tanh)
```

```
ModelTanh(
  (fc1): Linear(in_features=4, out_features=8, bias=True)
  (fc2): Linear(in_features=8, out_features=9, bias=True)
  (out): Linear(in_features=9, out_features=3, bias=True)
)
```



Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

ReLU Activation

```
class ModelReLU(nn.Module):
    #Input layer (4 features of the flower)
    #Hidden Layer 1 (number of neurons)
    #H2 (n)
    #Output (3 Classes of IRIS)
    def __init__(self, in_features=4, h1=8, h2=9, out_features=3):
        super().__init__() #initiate our nn model
        self.fc1 = nn.Linear(in_features, h1)
        self.fc2 = nn.Linear(h1, h2)
        self.out = nn.Linear(h2, out_features)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.out(x)
        return x
```

```
torch.manual_seed(32)
model_relu = ModelReLU()
print(model_relu)
```

```
ModelReLU(
  (fc1): Linear(in_features=4, out_features=8, bias=True)
  (fc2): Linear(in_features=8, out_features=9, bias=True)
  (out): Linear(in_features=9, out_features=3, bias=True)
)
```

Define the Dataset (IRIS Data)

Load the Iris dataset using sklearn and create a pandas DataFrame.

```
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target)
X.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2



Artificial Intelligence BS(AI)
Shaheed Zulfiqar Ali Bhutto Institute of Science and Technology
(SZABIST)

Convert Features to FloatTensor and Labels to LongTensor for Training and Testing

PyTorch requires data to be in a specific format called a tensor to perform computations.

- **Features:** The feature data (like sepal length and petal width) are numerical and should be converted to a torch.FloatTensor. This data type is used for floating-point numbers, which is necessary for most mathematical operations within the network.
- **Labels:** The labels (the species, in this case: 0, 1, 2) are integers representing discrete classes. These must be converted to a torch.LongTensor. This is a requirement for PyTorch's CrossEntropyLoss function, which expects the class labels to be of this integer type.

```
# Convert features to a FloatTensor (PyTorch models expect tensors, not DataFrames)
X_tensor = torch.FloatTensor(X.values)
X_tensor[:5]

tensor([[5.1000, 3.5000, 1.4000, 0.2000],
        [4.9000, 3.0000, 1.4000, 0.2000],
        [4.7000, 3.2000, 1.3000, 0.2000],
        [4.6000, 3.1000, 1.5000, 0.2000],
        [5.0000, 3.6000, 1.4000, 0.2000]])
```

Running the Models

The models are not trained, instead, the same 5 sample flowers from the Iris dataset are passed through all three models (Sigmoid, Tanh, and ReLU) using their randomly-initialized weights, and the raw outputs are compared.

This step demonstrates how the choice of activation function shapes the values flowing through the network, even before any learning takes place. It is not meant to produce accurate predictions.

Analysis and Comparison

Comparing the Activation Functions

Compare the shape and range of each activation function and relate it back to the plot generated earlier in the notebook:

- **Sigmoid** squashes any input into the range (0, 1). Its output is always positive, and it saturates (flattens) for large positive or negative inputs, which is why it can suffer from the vanishing gradient problem during training.
- **Tanh** squashes any input into the range (-1, 1) and is zero-centered, unlike Sigmoid. It also saturates for large inputs, but tends to produce more balanced gradients than Sigmoid.
- **ReLU** outputs 0 for any negative input and passes positive inputs through unchanged. It does not saturate on the positive side, is computationally cheap, and is the most widely used activation function in modern hidden layers.



Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

```
sample = X_tensor[:5]

with torch.no_grad():
    out_sigmoid = model_sigmoid(sample)
    out_tanh = model_tanh(sample)
    out_relu = model_relu(sample)

print('Sigmoid model output:\n', out_sigmoid)
print('\nTanh model output:\n', out_tanh)
print('\nReLU model output:\n', out_relu)
```

```
Sigmoid model output:
tensor([[ 0.1862,  0.3240, -0.6258],
        [ 0.1876,  0.3243, -0.6256],
        [ 0.1882,  0.3253, -0.6260],
        [ 0.1881,  0.3255, -0.6270],
        [ 0.1864,  0.3245, -0.6263]])
```

```
Tanh model output:
tensor([[ -0.2537, -0.2381, -0.0376],
        [-0.2405, -0.2413, -0.0337],
        [-0.2406, -0.2332, -0.0346],
        [-0.2425, -0.2325, -0.0477],
        [-0.2543, -0.2339, -0.0413]])
```

```
ReLU model output:
tensor([[ 0.1147,  0.0129, -0.1887],
        [ 0.1117,  0.0168, -0.2029],
        [ 0.1115,  0.0186, -0.1894],
        [ 0.1119,  0.0166, -0.1979],
        [ 0.1149,  0.0131, -0.1836]])
```

By comparing the plotted curves and the raw model outputs, you can observe how each activation function shapes the signal passed between layers, even without any training taking place.

Lab Tasks:

- Add a fourth activation function, **Leaky ReLU** (F.leaky_relu), to the activation function plot alongside Sigmoid, Tanh, and ReLU.
- Build a fourth model class, ModelLeakyReLU, using Leaky ReLU in its hidden layers, and add it to the output comparison on the 5 sample flowers.
- Explain how Leaky ReLU's shape differs from ReLU and what problem it is designed to solve.

Submission:

- Submit the Colab notebook with all steps executed.
- Ensure the notebook is well-commented and demonstrates your understanding of each operation.